

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz tartozó műveletek
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtovábbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A *logikai* adattípus

ABSZTRAKT adattípus

- Értékkészlete
 - $\{Hamis, Igaz\}$
- Műveletei (alapműveletek)
 - tagadás ($\neg$), és ($\wedge$), vagy ($\vee$), implikáció ($\rightarrow$), ...



A `bool` (*logikai*) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt *logikai* adattípus megvalósítása Python-ban.
- A `bool` típus értékkészlete a `False` és `True` értékekből áll.
- A legtöbb típusú érték automatikusan konvertálható `bool` típusúvá Python-ban: az üres vagy 0 értékek `False`, az egyéb értékek `True` logikai értéké.
- A `bool` *immutable* típus.



Az egész adattípus

ABSZTRAKT adattípus

- Értékkészlete
 - $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$
- Műveletei (alpműveletek)
 - negáció ($-$), összeadás ($+$), kivonás ($-$), szorzás ($\cdot$, $\times$), egészosztás ($/$), maradékképzés (mod)
 - összehasonlítások ($=$, $<$, $\leq$, $>$, $\geq$)



Az int (egész) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt egész adattípus megvalósítása Python-ban.
- Értékkészlete az összes egész szám, (elméletben) nincs korlátozva.
- Az `int` *immutable* típus.



A *valós* adattípus

ABSZTRAKT adattípus

- Értékkészlete
 - $\mathbb{R} = (-\infty, +\infty)$
- Műveletei (alapműveletek)
 - negáció ($-$), összeadás ($+$), kivonás ($-$), szorzás ($\cdot$, $\times$), osztás ($/$)
 - összehasonlítások ($=$, $<$, $\leq$, $>$, $\geq$)



A float (*valós*) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt *valós* adattípus megvalósítása Python-ban.
- Értékkészlete a $[-(2 - 2^{-52}) \cdot 2^{1023}, (2 - 2^{-52}) \cdot 2^{1023}]$ zárt intervallumba eső valós számok részhalmaza úgy, hogy ebből az intervallumból minden valós szám egy adott e relatív pontossággal megközelíthető egy `float` adattípusbeli értékkel.
- Az értékkészlet tartalmazza még a $-\text{inf}$ és $+\text{inf}$ értékeket is.
- Ez azt jelenti, hogy (az értékkészlet határértékein belül) bármely a valós számhoz van olyan x `float` típusú érték, hogy

$$\left| \frac{x - a}{a} \right| \leq e.$$

- A `float` *immutable* típus.

Tartalomjegyzék

- 
- 1 Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok**
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A *komplex* adattípus

ABSZTRAKT adattípus

- Értékkészlete

- $\mathbb{C} = \{re + imi | re, im \in \mathbb{R}, i = \sqrt{-1}\}$

- Műveletei (alpműveletek)

- negáció ($-$), összeadás ($+$), kivonás ($-$), szorzás ($\cdot$, $\times$), osztás ($/$), konjugált ($\bar{c}$), abszolútérték ($|c|$)
 - összehasonlítás ($=$)



A complex (*komplex*) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt *komplex* adattípus megvalósítása Python-ban.
 - A matematikai komplex számok $a + bi$ alakban (is) felírható számok, ahol $i = \sqrt{-1}$.
 - Bármely komplex szám bármely gyöke szintén komplex szám (valós számok esetében a negatív számok gyöke nem (mindig) értelmezett).
- Értékkészlete a valós típuson alapszik (felfogható egy kétdimenziós valós vektorként).
- A `complex` *immutable* típus.



A tömb adattípus

ABSZTRAKT adattípus

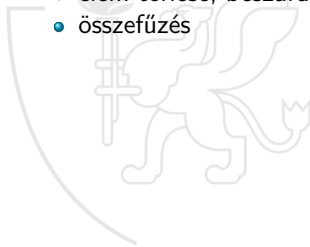
- Értékkészlete
 - $\mathbb{T}^n = \{t_0 t_1 \dots t_{n-1}\}$, ahol $\mathbb{T}$ egy tetszőleges típus és $n \in \mathbb{N}$.
- Műveletei (alpműveletek)
 - elem elérése (kiolvasása/módosítása)
 - összehasonlítás (=)



A *lista* adattípus

ABSZTRAKT adattípus

- Értékkészlete
 - $\mathbb{T}^* = \{t_0 t_1 \dots t_{n-1} \mid \forall i : 0 \leq i < n, t_i \in \mathbb{T}; n \in \mathbb{N}_0\}$, ahol $\mathbb{T}$ egy tetszőleges típus.
- Műveletei (alpműveletek)
 - elem elérése (kiolvasása/módosítása)
 - összehasonlítás (=)
 - hossz meghatározása
 - elem törlése, beszúrása index szerint
 - összefűzés



A list (*lista*) adattípus

Python VIRTUÁLIS adattípus

- A *listák* elemek (pozíció szerint) rendezett sorozatai.
- A `list` *mutable* típus: az elemeit megváltoztathatjuk, törölhetjük, beszúrhatunk elemeket, stb.

```
>>> l = [1, None, "Hello", (4, 5), ["Lista", "a", "listában"]]
>>> l.append(2024)
>>> del l[2]
>>> l
[1, None, (4, 5), ["Lista", "a", "listában"], 2024]
>>> l[0] = 0
>>> l[-2] = "A_végéről_a_második"
>>> l
[0, None, (4, 5), "A végéről a második", 2024]
```

A tuple adattípus

Python VIRTUÁLIS adattípus

- A *tuple*-ök elemek (pozíció szerint) rendezett sorozatai.
- A `tuple` *immutable* típus: ha egyszer létrehoztunk egy tuple-t, nem tudjuk megváltoztatni.
- A `tuple` típus tulajdonképpen egy *immutable list*.
- A `tuple` ennek megfelelően nem rendelkezik a `list` olyan műveleteivel, amelyek megváltoztathatnák az értékét.

```
>>> t = (1, None, "Hello", (4, 5), ["Lista", "a", "tuple-ben"])
>>> del t[2]
TypeError: 'tuple' object doesn't support item deletion
>>> t[2]
'Hello'
```

Másodfokú egyenlet gyökei

Problémafelvetés és specifikáció

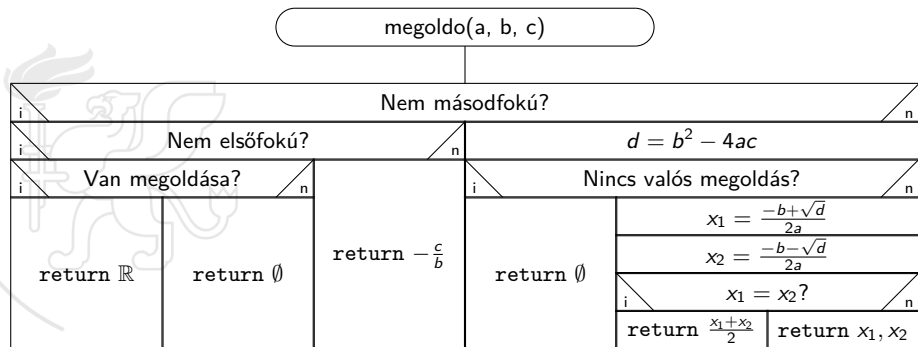
- Problémafelvetés
 - Határozzuk meg egy másodfokú egyenlet valós gyökeit!
- Specifikáció
 - A probléma inputja három valós szám, az $ax^2 + bx + c = 0$ másodfokú egyenlet a , b és c együtthatói.
 - Az output egy vagy két valós szám, az egyenlet valós gyöke vagy gyökei, különben – ha nincs valós gyök, vagy végtelen sok van – egy szöveg, ami a valós gyökök kiszámíthatatlanságát jelzi.



Másodfokú egyenlet gyökei

Algoritmustervezés

- A probléma megoldása olyan függvénnyel adható meg, amelynek
 - három bemenő paramétere van: a , b , c , az egyenlet együtthatói;
 - a visszetérési értéke a valós gyökök számától függ.
- A közismert $x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$ megoldóképletet használjuk.



Másodfokú egyenlet gyökei [1/3]

masodfok.py [1–10]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTIK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-06-04
6cc-by-nc-nd
7
8Másodfokú egyenlet valós gyökeinek meghatározása.
9"""
10from math import sqrt
```



Másodfokú egyenlet gyökei [2/3]

masodfok.py [12–31]

```
12 def megoldo(a, b, c):
13     if a == 0.0:
14         if b == 0.0:
15             if c == 0.0:
16                 return True
17             else:
18                 return None
19         else:
20             return -(c / b)
21     else:
22         d = b ** 2 - 4.0 * a * c
23         if d < 0.0:
24             return None
25         else:
26             x1 = (-b + sqrt(d)) / (2.0 * a)
27             x2 = (-b - sqrt(d)) / (2.0 * a)
28             if abs(x1 - x2) < 1e-10:
29                 return (x1 + x2) / 2.0
30             else:
31                 return (x1, x2)
```

Másodfokú egyenlet gyökei [3/3]

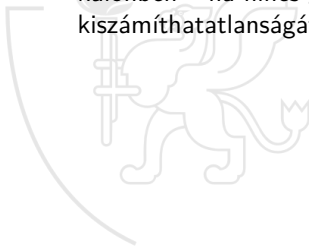
masodfok.py [33–49]

```
33 def main():
34     print("Kérem az egyenlet együtthatóit!")
35     a = float(input("a? "))
36     b = float(input("b? "))
37     c = float(input("c? "))
38     gyok = megoldo(a, b, c)
39     if gyok is None:
40         print("Nincs valós gyök!")
41     elif gyok is True:
42         print("Végtelen sok megoldás van.")
43     elif type(gyok) is tuple:
44         print("A két valós gyök:", gyok[0], "és", gyok[1])
45     else:
46         print("Az egyetlen valós gyök:", gyok)
47
48 if __name__ == "__main__":
49     main()
```

Másodfokú egyenlet gyökei

Problémafelvetés és specifikáció

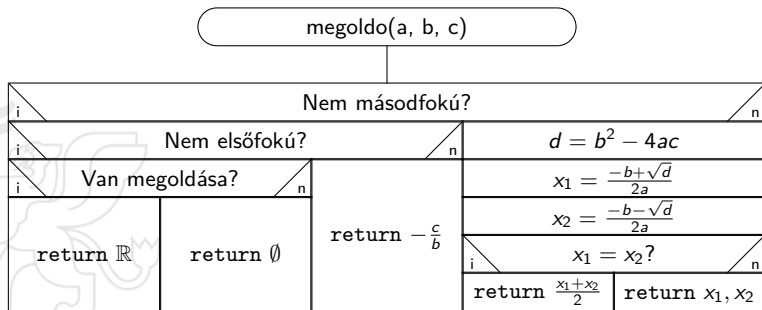
- Problémafelvetés
 - Határozzuk meg egy másodfokú egyenlet komplex gyökeit!
- Specifikáció
 - A probléma inputja három valós szám, az $ax^2 + bx + c = 0$ másodfokú egyenlet a , b és c együtthatói.
 - Az output két komplex szám, az egyenlet gyökei, vagy egy valós szám, az egyetlen gyök, különben – ha nincs gyök, vagy végtelen sok van – egy szöveg, ami a gyökök kiszámíthatatlanságát jelzi.



Másodfokú egyenlet gyökei

Algoritmustervezés

- A probléma megoldása hasonló a valós gyököt számító megoldáshoz, csak most a komplex gyökvonás miatt nem kell ellenőriznünk, hogy negatív-e a megoldóképlet gyök alatti része.



Másodfokú egyenlet gyökei [1/2]

masodfok_c.py [1-28]

```
1#!/usr/bin/env python3
2"""
3Programozás, Alapjai, előadás, példaprogram
4SZTE, TTIK, Informatikai Intézet, Szoftverfejlesztés, Tanszék
5Gergely, Tamás, gertom@inf.u-szeged.hu, 2024-06-04
6cc-by-nc-nd
7
8Másodfokú egyenlet gyökeinek meghatározása.
9"""
10from cmath import sqrt
11
12def megoldo(a, b, c):
13    if a == 0.0:
14        if b == 0.0:
15            if c == 0.0:
16                return True
17            else:
18                return None
19        else:
20            return -(c / b)
21    else:
22        d = b ** 2 - 4.0 * a * c
23        x1 = (-b + sqrt(d)) / (2.0 * a)
24        x2 = (-b - sqrt(d)) / (2.0 * a)
25        if abs(x1 - x2) < 1e-10:
26            return (x1 + x2) / 2.0
27        else:
28            return (x1, x2)
```

Másodfokú egyenlet gyökei [2/2]

masodfok_c.py [30–46]

```
30 def main():
31     print("Kérem az egyenlet együtthatóit!")
32     a = float(input("a? "))
33     b = float(input("b? "))
34     c = float(input("c? "))
35     gyok = megoldo(a, b, c)
36     if gyok is None:
37         print("Nincs gyök!")
38     elif gyok is True:
39         print("Végtelen sok megoldás van.")
40     elif type(gyok) is tuple:
41         print("A két gyök:", gyok[0], "és", gyok[1])
42     else:
43         print("Az egyetlen gyök:", gyok)
44
45 if __name__ == "__main__":
46     main()
```

- Problémafelvetés

- Egy város gyógyszerértárai között riadóláncot kell szervezni. Minden gyógyszerértárra meghatározzuk, hogy sürgős esetben (pl. gyógyszer azonnali visszavonása esetén) kit kell értesítenie. Eldöntendő, hogy adott értesítési hozzárendelések valóban riadóláncot alkotnak-e, azaz bárki is indítja az üzenetet, az az értesítéseken keresztül minden gyógyszerértárba el fog-e jutni?

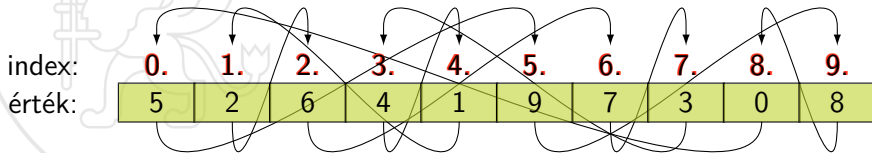
- Specifikáció

- A probléma inputja egy n szám, a gyógyszerértárak (amiket a $[0, \dots, n - 1]$ sorszámokkal azonosítunk) száma, valamint egy n elemű számsorozat, melynek i . tagja az i . sorszámú gyógyszerértár által értesítendő másik gyógyszerértár sorszáma.
- Az output egy „Igen” vagy „Nem” tetszőleges szövegkörnyezetben.

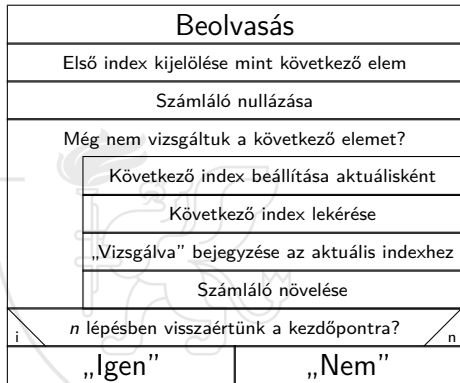
Riadólánc

Algoritmustervezés

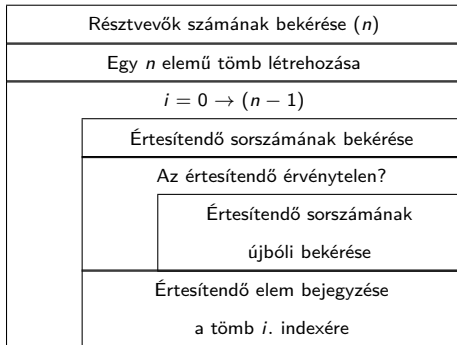
- A riadólánc egy tömbbel ábrázolható, ahol a tömb i . elemének értéke annak a gyógyszerértéknek a sorszáma, amelyiket az i gyógyszerértéknek kell értesítenie.
- A tömb akkor és csak akkor valódi riadólánc, ha *bármelyik* i elemétől elindulva és a tömb szerinti hozzárendelést követve visszajutunk i -be úgy, hogy közben minden $[0, \dots, n - 1]$ intervallumba eső elemet érintettünk.
- Ezt azonban elég egy tetszőleges elemtől kezdve kipróbálni.



- A program struktogramja:



- A beolvasás struktogramja:



Riadólánc [1/2]

riadolanc.py [1-22]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-06-04
6cc-by-nc-nd
7
8Adott egy község, értesítési relációkkal. Eldöntendő,
9hogyan riadóláncot alkotnak-e?
10"""
11ELSO = 0
12
13def main():
14    global ELSO
15
16    n = int(input("Hány elemű a sorozat? "))
17    lanc = [None] * n
18    for i in range(n):
19        e = int(input("A(z) %d. tag kit értesít? " % (i)))
20        while (e < 0) or (n <= e):
21            e = int(input("Hibás adat, kérem újra: "))
22    lanc[i] = e
```

Riadólánc [2/2]

riadolanc.py [24–38]

```
24     kov = ELSONY
25     szam = 0
26     while lanc[kov] is not None:
27         i = kov
28         kov = lanc[i]
29         lanc[i] = None
30         szam += 1
31
32     if szam == n and kov == ELSONY:
33         print("A számsorozat riadólánc.")
34     else:
35         print("A számsorozat nem riadólánc.")
36
37 if __name__ == "__main__":
38     main()
```

- Problémafelvetés

- A *Riadólánc* algoritmusban a bemenő adatot tároló tömb az algoritmus végrehajtása során módosul. Ez elkerülendő akkor, ha a bemenő adaton más műveletet is kell végeznünk.
- Készítsünk tehát egy olyan függvényt, amely egy értesítési láncról eldönti, hogy az riadólánc-e (matematikai megfogalmazással: eldönti, hogy a kapott adatok ciklikus permutációt határoznak-e meg) anélkül, hogy a láncot magát megváltoztatná.

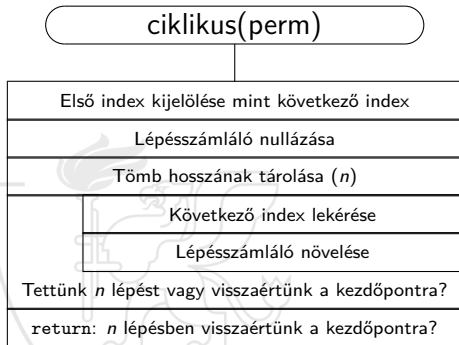
- Specifikáció

- A függvény inputja egy n elemű tömb, melynek i . tagja az i . sorszámú gyógyszerár által értesítendő gyógyszerár sorszáma. A tömb bemenő paraméter, értékét a függvény nem változtathatja meg.
- Az output a függvény visszatérési értéke, egy logikai igaz érték ha az input ciklikus permutáció, illetve hamis ha nem az.

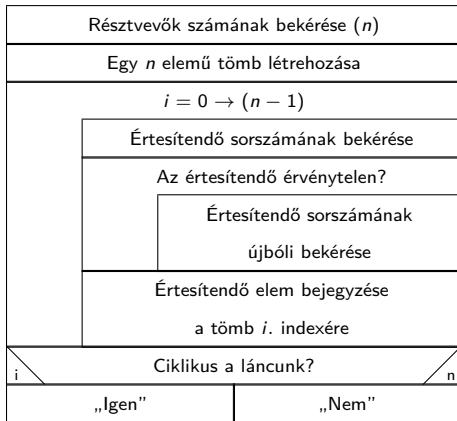
Ciklikus függvény

Algoritmustervezés – Struktogram

- A ciklikus függvény struktogramja:



- A program struktogramja:



Riadólánc

riadolanc_ciklikus.py [1–25]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-06-04
6cc-by-nc-nd
7
8Adott egy közösség, értesítési relációkkal. Eldöntendő,
9hogyan lehet láncot alkotni, úgy, hogy közben ne rontsuk el a láncot.
10"""
11ELSO = 0
12
13def ciklikus(perm):
14    global ELSO
15
16    kov = ELSO
17    szam = 0
18    n = len(perm)
19
20    while True:
21        kov = perm[kov]
22        szam += 1
23        if kov == ELSO or szam == n:
24            break
25    return kov == ELSO and szam == n
```

Riadólánc

riadolanc_ciklikus.py [27–39]

```
27 def main():
28     n = int(input("Hány elemű a sorozat? "))
29     lanc = [None] * n
30     for i in range(n):
31         e = int(input("A(z) %d. tag kit értesít? " % (i)))
32         while (e < 0) or (n <= e):
33             e = int(input("Hibás adat, kérem újra: "))
34         lanc[i] = e
35
36     if ciklikus(lanc):
37         print("A számsorozat riadólánc.")
38     else:
39         print("A számsorozat nem riadólánc.")
```